

Learning LlamaIndex: The 20% That Matters

1 What LlamaIndex Is (and How It Relates to the Other Tools)

LlamaIndex is a **data framework** for LLM apps. It is the high-level, batteries-included way to do everything in the RAG document: it handles loading, chunking, embedding, storing, retrieving, and prompting for you. Where the RAG document built the pipeline by hand, LlamaIndex collapses it into a few lines.

Positioning vs the other tools:

- **vs hand-built RAG** — same concepts, far less boilerplate; you drop to the low-level pieces only when you need control.
- **vs LangGraph** — LangGraph orchestrates arbitrary agent control flow; LlamaIndex specializes in connecting LLMs to your *data*. For pure RAG, LlamaIndex usually gives a cleaner developer experience. They are often used together.

2 The Famous 5-Line RAG

This is the entire quickstart, and it really is this short:

```
1 from llama_index.core import VectorStoreIndex, SimpleDirectoryReader
2
3 documents = SimpleDirectoryReader("data").load_data()      # 1. load
4 index = VectorStoreIndex.from_documents(documents)          # 2. chunk+embed+
   store
5 query_engine = index.as_query_engine()                     # 3. make a query
   engine
6 response = query_engine.query("What did the author do growing up?") # 4.
   ask
7 print(response)                                           # 5. answer
```

Under the hood this loaded your files, split them into nodes, embedded each node, stored the vectors in memory, and on query: embedded the question, retrieved the top nodes, and synthesized an answer. The rest of this document is about controlling each of those steps.

3 The Core Vocabulary

- **Document** — a loaded source file (text + metadata).
- **Node** — a chunk of a Document; the unit that gets embedded and retrieved.
- **Index** — the structure holding your nodes (usually `VectorStoreIndex`).
- **Query Engine** — ask a question, get a synthesized answer (RAG in one object).

- **Chat Engine** — like a query engine but with conversation memory.
- **Retriever** — the low-level piece that returns raw nodes, no answer synthesis.

4 Setup

```
1 pip install llama-index
2 # Pieces are modular; install integrations as needed, e.g.:
3 pip install llama-index-embeddings-huggingface llama-index-vector-stores-chroma
```

5 Loading Data

`SimpleDirectoryReader` reads a whole folder (PDF, docx, txt, md, and more).

```
1 from llama_index.core import SimpleDirectoryReader
2
3 docs = SimpleDirectoryReader("data", recursive=True).load_data()
4 # recursive=True walks subfolders. LlamaHub has hundreds of other loaders
5 # (Notion, Slack, databases, web pages, etc.)
```

6 Global Configuration with Settings

`Settings` is the global config object (it replaced the older `ServiceContext`). Set your LLM, embedding model, and chunk size once.

```
1 from llama_index.core import Settings
2 from llama_index.embeddings.huggingface import HuggingFaceEmbedding
3
4 Settings.embed_model = HuggingFaceEmbedding(model_name="BAAI/bge-small-en-v1.5")
5 Settings.chunk_size = 512 # node size in tokens
6 Settings.chunk_overlap = 50
7 # Settings.llm = ... # defaults to OpenAI; set any LLM here
```

Why this matters: the embedding model decides retrieval quality (see the RAG doc), and you must use the same one for indexing and querying — setting it globally guarantees that.

7 Building the Index (with Chunking Control)

You can override chunking per-index via `transformations`:

```
1 from llama_index.core import VectorStoreIndex
2 from llama_index.core.node_parser import SentenceSplitter
3
4 index = VectorStoreIndex.from_documents(
5     docs,
6     transformations=[SentenceSplitter(chunk_size=512, chunk_overlap=50)],
7 )
```

8 Persisting the Index (Stop Re-Embedding Every Run)

By default the index lives in memory and is lost when the program ends — re-embedding on every run wastes time and money. Persist it once, then load it.

```
1 # Save
2 index.storage_context.persist(persist_dir="./storage")
3
4 # Load later
5 from llama_index.core import StorageContext, load_index_from_storage
6 storage = StorageContext.from_defaults(persist_dir="./storage")
7 index = load_index_from_storage(storage)
```

9 The Query Engine (Your Main Tool)

```
1 qe = index.as_query_engine(
2     similarity_top_k=5,           # how many nodes to retrieve
3     response_mode="compact",     # how retrieved text is combined into the
4     streaming=True,              # stream the answer token by token
5 )
6 response = qe.query("Summarize the onboarding process.")
7 print(response)                 # the synthesized answer
8 print(response.source_nodes)    # the chunks it used -- inspect these to
                                # debug!
```

Always inspect `source_nodes` when an answer looks wrong — it tells you whether retrieval (wrong chunks) or generation (wrong use of chunks) is the problem.

10 The Chat Engine (Adds Memory)

```
1 chat = index.as_chat_engine(chat_mode="context")
2 print(chat.chat("What is our refund policy?"))
3 print(chat.chat("And how long does it take?")) # remembers the prior
4                                                  # turn
```

11 The Retriever (When You Only Want Chunks)

If you want the raw nodes (e.g. to feed your own prompt, or rerank yourself):

```
1 retriever = index.as_retriever(similarity_top_k=10)
2 nodes = retriever.retrieve("password reset")
3 for n in nodes:
4     print(n.score, n.text[:80])
```

12 Using a Real Vector Store

In-memory is fine for learning; for anything persistent or large, plug in a vector store like Chroma (same database from the RAG doc).

```

1 import chromadb
2 from llama_index.vector_stores.chroma import ChromaVectorStore
3 from llama_index.core import StorageContext
4
5 chroma = chromadb.PersistentClient(path="./chroma_db")
6 collection = chroma.get_or_create_collection("docs")
7 vector_store = ChromaVectorStore(chroma_collection=collection)
8 storage = StorageContext.from_defaults(vector_store=vector_store)
9
10 index = VectorStoreIndex.from_documents(docs, storage_context=storage)

```

13 Improving Quality: Reranking (Postprocessors)

LlamaIndex lets you add a reranker (the two-stage retrieve-then-rerank idea from the RAG doc) as a **node postprocessor**:

```

1 from llama_index.core.postprocessor import SentenceTransformerRerank
2
3 reranker = SentenceTransformerRerank(model="cross-encoder/ms-marco-MiniLM-
4   L-6-v2", top_n=3)
5 qe = index.as_query_engine(similarity_top_k=20, node_postprocessors=[
6   reranker])
7 # retrieves 20, reranks, keeps the best 3 for the prompt

```

14 Turning RAG into a Tool for an Agent

A query engine can become a **tool** an agent calls. This is how LlamaIndex builds agents: wrap each capability as a tool, hand them to an agent.

```

1 from llama_index.core.tools import QueryEngineTool, FunctionTool
2
3 doc_tool = QueryEngineTool.from_defaults(
4     index.as_query_engine(),
5     name="document_search",
6     description="Search internal company documents.", # the agent reads
7     this
8 )
9
10 def get_weather(city: str) -> str:
11     """Return the weather for a city."""
12     return f"Sunny in {city}"
13
14 weather_tool = FunctionTool.from_defaults(fn=get_weather)

```

Then give the tools to an agent. **Version note:** LlamaIndex's agent class has changed names across versions (older ReActAgent/OpenAIAgent; current builds favor FunctionAgent / AgentWorkflow from llama_index.core.agent.workflow). Check the docs for your installed version. The *tool* creation above is stable.

```

1 from llama_index.core.agent.workflow import FunctionAgent
2

```

```

3 agent = FunctionAgent(tools=[doc_tool, weather_tool], llm=Settings.llm)
4 response = await agent.run("Search the docs for the refund policy.")

```

15 Router (Awareness)

A `RouterQueryEngine` picks among several engines based on the question — e.g. a vector index for specific facts and a summary index for “summarize everything” questions. Each sub-engine is described in plain text and the router chooses. Useful once you have multiple data sources.

16 When to Use What

- **Quick document QA** → `as_query_engine()`.
- **Conversational** → `as_chat_engine()`.
- **Custom prompt / your own rerank** → `as_retriever()`.
- **Tool-using / multi-step** → wrap engines as tools, use an agent.
- **Complex custom control flow** → orchestrate with `LangGraph`, using `LlamaIndex` for the data/retrieval pieces.

17 The Checklist for Any LlamaIndex App

1. Set `Settings` (embedding model, chunk size, LLM) once.
2. Load data with `SimpleDirectoryReader` (or a `LlamaHub` loader).
3. Build a `VectorStoreIndex`; **persist** it so you don’t re-embed.
4. Pick the interface: query engine, chat engine, or retriever.
5. Tune `similarity_top_k`; add a reranker for quality.
6. Inspect `source_nodes` to debug retrieval.
7. For agents, wrap engines/functions as tools.

18 Practice Task

1. Put 3–4 text/PDF files in a `data/` folder.
2. Set `Settings.embed_model` to a Hugging Face model and build a `VectorStoreIndex`.
3. Persist the index, then in a fresh run load it from `./storage` (confirm no re-embedding).
4. Query it; print both the answer and `response.source_nodes`.
5. Add a `SentenceTransformerRerank` postprocessor and compare answers.
6. Wrap the query engine as a `QueryEngineTool`, add one `FunctionTool`, and run an agent that uses both.
7. Bonus: expose it via a FastAPI `POST /ask` endpoint, in Docker, with deps managed by uv.